

Tutorial 01: First Spawn and Harvester

A practical local-server Screeps module

<docs/tutorials/01-first-spawn-and-harvester.md>

Episode 1: Touchdown

The hatch is open. The room is quiet. Your collaborator is still narrating the scan results, but the console is yours now – welcome to Day 1, no orientation packet included.

Somewhere else on this planet, other Foundry architects are doing exactly what you're about to do: picking a room, placing a spawn, and watching whether their first worker can complete one single lap without dying, getting stuck, or just standing there like it's waiting on a code review. Nobody skips this step. Nobody gets to – not even the ones who've shipped to production for a decade.

This is the first system you'll ship. Keep it small. Keep it visible. Everything after this builds on whatever you learn watching it run, so resist the urge to get clever before you've even proven the loop works.

Goal

Get from a fresh local Screeps world to a working first room with:

- One selected room
- One placed spawn
- One manually spawned worker
- One basic script loop that harvests energy and returns it to the spawn

This tutorial is for the local AutoNateAI Screeps server at `localhost:21025`.

Prerequisites

The local server is running:

```
docker ps --filter name=autonate-screeps
```

The Screeps: World client is connected to:

```
Host: localhost
Port: 21025
Server password: blank
```

The local profile exists:

```
Username: autonate
Password: 1234
```

In this setup the Steam client is mapped to `autonate` by `SCREEPS_LOCAL_DEFAULT_USER=autonate`, so the client may not ask for the password.

Step 1: Choose a Room

On the world map, pick an unowned room.

For this first run, prefer a room that has:

- Two visible energy sources
- A controller
- Enough open area near the sources and controller
- No obvious hostile owned structures

Click the room and enter it.

Checkpoint:

- You can see terrain, sources, and a room controller.
- The room owner should be `None` before you claim/place your spawn.

Step 2: Place the First Spawn

Use the client prompt to place your first spawn.

Name it:

```
Spawn1
```

Placement guidance:

- Put it near open terrain.
- Do not place it directly against too many walls.
- Being somewhat close to at least one energy source helps early movement.
- Do not optimize too hard. This first spawn is for learning the loop.

Checkpoint:

- The room now has a spawn named `Spawn1`.
- The top-right user menu shows you are connected as `autonate`.

 **Screenshot placeholder:** `Spawn1` freshly placed in the client, before any creep exists – this is the visual "checkpoint" a new reader can compare their own room against.

Step 3: Create One Harvester Manually

Open the in-game console and run:

```
Game.spawns[ 'Spawn1' ].spawnCreep([WORK, CARRY, MOVE], 'Harvester1');
```

Expected result:

```
0
```

If you get `0`, the spawn accepted the request.

Common results:

- `0`: OK
- `-4`: spawn is busy
- `-6`: not enough energy
- `-3`: name already exists

Checkpoint:

- A creep named `Harvester1` appears after spawning completes.

Step 4: Add the First Script

Open the in-game script editor.

In `main`, replace the contents with:

```
module.exports.loop = function () {  
  const creep = Game.creeps.Harvester1;  
  
  if (!creep) {  
    return;  
  }  
  
  if (creep.store.getFreeCapacity() > 0) {
```

```
const sources = creep.room.find(FIND_SOURCES);
if (creep.harvest(sources[0]) === ERR_NOT_IN_RANGE) {
  creep.moveTo(sources[0], { visualizePathStyle: { stroke: '#ffaa00' } });
}
return;
}

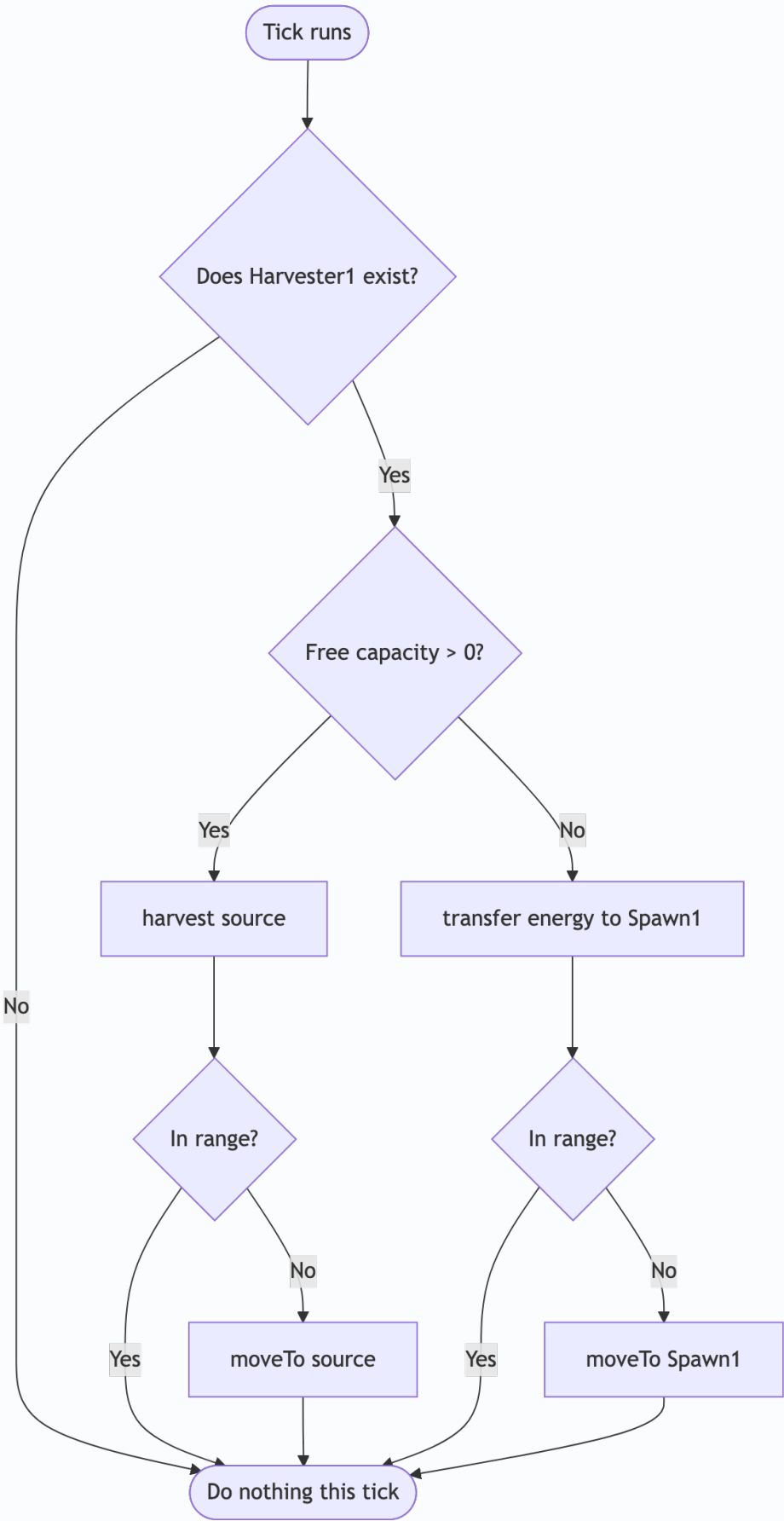
const spawn = Game.spawns.Spawn1;
if (creep.transfer(spawn, RESOURCE_ENERGY) === ERR_NOT_IN_RANGE) {
  creep.moveTo(spawn, { visualizePathStyle: { stroke: '#ffffff' } });
}
};
```

Save the script.

Checkpoint:

- Harvester1 moves to a source.
- It harvests until full.
- It returns to Spawn1.
- It transfers energy into the spawn.

This is the decision your script re-runs every single tick – the entire loop, drawn out:



Every role in every later episode is a variation on this exact shape: check state, act or move closer, repeat next tick.

Step 5: Observe the Loop

Watch for 2-3 full trips.

Record:

- How long the creep takes to reach the source
- Whether it gets stuck
- Whether it returns energy correctly
- Whether the spawn sits idle
- Whether the path seems wasteful

This is not about writing perfect code yet. It is about seeing the game loop clearly.

Step 6: Add Auto-Respawn

Now replace `main` with this slightly better version:

```
module.exports.loop = function () {
  for (const name in Memory.creeps) {
    if (!Game.creeps[name]) {
      delete Memory.creeps[name];
    }
  }

  if (!Game.creeps.Harvester1 && !Game.spawns.Spawn1.spawning) {
    Game.spawns.Spawn1.spawnCreep([WORK, CARRY, MOVE], 'Harvester1');
  }

  const creep = Game.creeps.Harvester1;
  if (!creep) {
    return;
  }

  if (creep.store.getFreeCapacity() > 0) {
    const sources = creep.room.find(FIND_SOURCES);
    if (creep.harvest(sources[0]) === ERR_NOT_IN_RANGE) {
      creep.moveTo(sources[0], { visualizePathStyle: { stroke: '#ffaa00' } });
    }
    return;
  }

  const spawn = Game.spawns.Spawn1;
  if (creep.transfer(spawn, RESOURCE_ENERGY) === ERR_NOT_IN_RANGE) {
    creep.moveTo(spawn, { visualizePathStyle: { stroke: '#ffffff' } });
  }
};
```

Checkpoint:

- If `Harvester1` dies, the spawn creates it again.
- Dead creep memory is cleaned up.

Troubleshooting

If nothing happens:

```
Game.spawns
```

Make sure the spawn is named `Spawn1`.

If the creep does not exist:

```
Game.creeps
```

If the spawn is busy:

```
Game.spawns.Spawn1.spawning
```

If the spawn has no energy:

```
Game.spawns.Spawn1.store[RESOURCE_ENERGY]
```

If the script has an error, check the console output in the client.

Completion Criteria

You are done when:

- You can connect to the local server.
- You have a room with `Spawn1`.
- `Harvester1` exists.
- `Harvester1` harvests from a source.
- `Harvester1` transfers energy to the spawn.
- The script can respawn `Harvester1` if it dies.

Learning Notes

After completing the tutorial, write down:

- What room did you choose?
- Where did you place `Spawn1`?
- What error codes did you see?
- What did the creep do that surprised you?
- What would make this script fail with two harvesters?
- What should the next tutorial add: upgrading, building, or multiple roles?

Next: Episode 2 – Two Sources, One Colony

Your collaborator flags it before you do: "Harvester1 only knows about `sources[0]`. The second you spawn a friend for it, they're going to fight over the same source and let the other one sit there untouched – real 'two founders, one laptop' behavior."

That's the problem Episode 2 picks up: teaching the colony to assign work instead of guessing at it.

See `docs/roadmap.md` for the full season.